



Curso ROS2 básico: Python

Aula 02 – Criando o Primeiro Nó

Objetivo

Aprender a criar um nó simples em Python utilizando `roscpp`, a biblioteca cliente do ROS 2 para Python. O estudante compreenderá a estrutura de um nó, como ele é inicializado, executado e encerrado.

1. O que é um Nó no ROS 2?

No ROS 2, um **nó** é uma unidade de execução que realiza uma tarefa específica dentro do sistema robótico. Pode ser um nó que publica dados, assina um tópico, oferece um serviço ou executa uma ação.

Cada nó tem um **nome** e é gerenciado pelo sistema de comunicação do ROS 2 (DDS).

2. Estrutura básica de um nó com `roscpp`

O `roscpp` é a biblioteca cliente do ROS 2 para a linguagem Python. A sigla significa **Robot Communication Library for Python**, ou seja, é a camada que permite criar, configurar e controlar nós ROS 2 em Python. Ela permite criar, configurar e executar nós ROS 2 diretamente em códigos Python, tornando o desenvolvimento mais rápido e acessível.

Componentes principais usados no exemplo:

- `rclpy` → biblioteca que inicializa e finaliza o ROS 2 no código Python
- `Node` → classe base para qualquer nó no ROS 2
- `get_logger()` → fornece um sistema de log padronizado do ROS 2
- `rclpy.spin()` → deixa o nó ativo, aguardando eventos ou comunicações
- `destroy_node()` → encerra o nó de forma segura
- `shutdown()` → encerra a sessão do ROS 2

Exemplo: nó que imprime "Hello ROS 2"

Crie o arquivo `hello_node.py` dentro da pasta do seu pacote:

```
cd ~/ros2_ws/src/meu_primeiro_pacote/meu_primeiro_pacote
nano hello_node.py
```

Conteúdo (com comentários):

```
# Importa a biblioteca cliente do ROS 2 para Python
import rclpy

# Importa a classe base para criação de nós
from rclpy.node import Node

# Define uma classe que representa o nó
class HelloNode(Node):
    def __init__(self):
        # Inicializa o nó com o nome 'hello_node'
        super().__init__('hello_node')
        # Envia uma mensagem para o terminal utilizando o logger do ROS 2
        self.get_logger().info('Hello ROS 2!')

# Função principal que inicializa e executa o nó
def main(args=None):
    # Inicializa o ambiente ROS 2
    rclpy.init(args=args)

    # Cria uma instância do nó
    node = HelloNode()

    # Mantém o nó em execução até que seja encerrado manualmente
    rclpy.spin(node)

    # Encerra o nó
    node.destroy_node()

    # Finaliza o ROS 2
    rclpy.shutdown()

# Executa a função principal quando o script é executado diretamente
```

```
if __name__ == '__main__':  
    main()
```

Verificando e executando nós com comandos do ROS 2

- Verificar se o nó está rodando:

```
ros2 node list
```

- Verificar informações detalhadas do nó:

```
ros2 node info /hello_node
```

- Verificar os logs (em tempo real):

```
ros2 run meu_primeiro_pacote hello_node
```

Você verá:

```
[INFO] [hello_node]: Hello ROS 2!
```

Dê permissão de execução ao arquivo:

```
chmod +x hello_node.py
```

4. Registrando o nó em `setup.py`

Para que o comando `ros2 run` funcione com o seu nó, é necessário **registrá-lo como um executável** no arquivo `setup.py` do seu pacote. Isso permite que o ROS 2 saiba qual script deve ser executado ao chamar o nome do nó.

Abra o arquivo `setup.py`:

```
nano ~/ros2_ws/src/meu_primeiro_pacote/setup.py
```

Localize o campo `entry_points` e edite da seguinte forma:

```
entry_points={  
    'console_scripts': [  
        'hello_node = meu_primeiro_pacote.hello_node:main'    ]  
}
```

```
],  
},
```

Explicando:

- 'hello_node' → é o nome do executável, ou seja, o nome usado no comando `ros2 run`
- `meu_primeiro_pacote.hello_node:main` → indica o caminho do script e a função que deve ser chamada:
 - `meu_primeiro_pacote.hello_node` → corresponde ao arquivo `hello_node.py` dentro da pasta do pacote
 - `main` → é a função que será executada

Esse registro transforma a função `main()` no ponto de entrada do seu nó.

Salve e feche o arquivo (`Ctrl+O`, `Enter`, `Ctrl+X`).

5. Recompilando o workspace

```
cd ~/ros2_ws  
colcon build --packages-select meu_primeiro_pacote  
source install/setup.bash
```

6. Executando o nó

```
ros2 run meu_primeiro_pacote hello_node
```

Se tudo estiver certo, você verá no terminal:

```
[INFO] [hello_node]: Hello ROS 2!
```

7. Explicação detalhada do código

```
import rclpy  
from rclpy.node import Node
```

Importamos a biblioteca `rclpy` e a classe base `Node`, que será usada para criar nosso nó personalizado.

```
class HelloNode(Node):  
    def __init__(self):
```

```
super().__init__('hello_node')
self.get_logger().info('Hello ROS 2!')
```

Aqui definimos uma classe chamada `HelloNode`, que herda de `Node`. Dentro do `__init__`, chamamos o construtor da superclasse passando o nome do nó: `'hello_node'`. Logo em seguida, usamos `get_logger().info()` para imprimir uma mensagem no terminal.

```
def main(args=None):
    rclpy.init(args=args)
    node = HelloNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

- `rclpy.init()` inicializa o sistema ROS 2
- Criamos uma instância da classe `HelloNode`
- `rclpy.spin()` mantém o nó ativo (mesmo que ele não tenha callbacks)
- `destroy_node()` encerra o nó de forma segura
- `shutdown()` finaliza o ambiente ROS 2

8. Exercício sugerido

Altere a mensagem impressa para incluir o seu nome. Exemplo:

```
Hello ROS 2! Aqui é o Clístenes.
```

Próxima aula: vamos aprender a publicar mensagens em tópicos com um publisher ROS 2.

MetaBee | Educação, Inovação e Robótica

